



Resumen clase 28/08 - Scheduling

Introducción a Scheduling

Objetivos

Tipos de scheduling

Scheduling apropiativo o con desalojo

Scheduling no apropiativo o con multitarea cooperativa

Conceptos clave

Rafaga CPU y Rafaga E/S

Flujo normal de rafagas

Procesos **CPU-bound**

Procesos **I/O-bound**

Métricas de rendimiento (performance metrics)

Inanición (Starvation)

Algoritmos clásicos

FCFS (First Come First Served).

Problema

Solución

Nuevo problema (Inanición/Starvation)

Posible solución

Round Robin (RR).

¿Cúanto dura el quantum?

Combinación con prioridades

SJF (Shortest Job First).

Duración total

Uso de CPU

SRTF (Shortest Remaining Time First).

Prioridades (con/sin desalojo).

Colas de multinivel o Múltiples colas (Multilevel Queue Scheduling).

Colas de multinivel con feedback (aging).

Scheduling en Tiempo Real

Tiempo real duro

Tiempo real suave

EDF (Earliest Deadline First).

Introducción a Scheduling

Objetivos

El principal objetivo del **Scheduling** es optimizar el rendimiento del **SO**.

- Que optimizar:
 - **Ecuanimidad (fairness)**: que cada proceso reciba una dosis "justa" de CPU (para alguna definición de justicia).
 - **Eficiencia**: tratar de que la CPU esté ocupada todo el tiempo.
 - **Carga del sistema**: minimizar la cant. de procesos listos que están esperando CPU.
 - **Tiempo de respuesta**: minimizar el tiempo de respuesta percibido por los usuarios interactivos.
 - **Latencia**: minimizar el tiempo requerido para que un proceso empiece a dar resultados.
 - **Tiempo de ejecución**: minimizar el tiempo total que le toma a un proceso ejecutar completamente.
 - **Rendimiento (throughput)**: maximizar el número de procesos terminados por unidad de tiempo.
 - **Liberación de recursos**: hacer que terminen cuanto antes los procesos que tiene reservados más recursos.

En definitiva, cada política de scheduling va a buscar **maximizar una función objetivo**, que va a ser una combinación de estas metas **tratando de impactar lo menos posible** en el **resto**.



Muchas de estas políticas son contradictorias

Tipos de scheduling

El **scheduling** puede ser **cooperativo** o **con desalojo**

Scheduling apropiativo o con desalojo

También llamado **scheduling apropiativo** o **preemptive**, el **scheduler** se vale de la interrupción del **clock** para decidir si el proceso debe seguir ejecutándose o le toca a otro, recordemos que el **clock interrumpe** de **50-60 veces por segundo**.

Un **scheduling con desalojo**:

- Requiere un clock con interrupciones.
- No le da garantía de continuidad a los procesos.
- También combinan su enfoque con el **cooperativo**.

Scheduling no apropiativo o con multitarea cooperativa

- Una vez que un proceso obtiene la CPU, **no se le quita hasta que termine su ráfaga** o llame a E/S voluntariamente.
 - El scheduler **no interrumpe** al proceso.
 - El scheduler analiza la situación cuando el kernel toma control (en los syscalls).
 - Especialmente cuando el proceso hace E/S.
 - A veces se proveen llamadas explícitas para permitir que se ejecuten otros procesos.
-

Conceptos clave

Ráfaga CPU y Ráfaga E/S

Casi todos los procesos alternan ráfagas de cálculos (CPU) con peticiones de E/S (de disco).

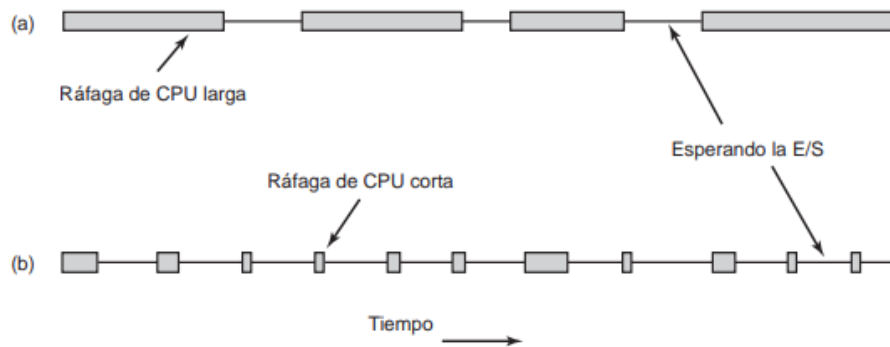


Figura 2-38. Las ráfagas de uso de la CPU se alternan con los periodos de espera por la E/S. (a) Un proceso ligado a la CPU. (b) Un proceso ligado a la E/S.

Flujo normal de rafagas

- Por lo general la CPU opera durante cierto tiempo sin detenerse.
- Después se realiza una llamada al sistema para leer datos de un archivo o escribirlos en el mismo.
- Cuando se completa la llamada al sistema, la CPU realiza cálculos de nuevo hasta que necesita más datos o tiene que escribir más datos y así sucesivamente.

Procesos CPU-bound

Largas ráfagas de CPU, poca E/S.

Procesos I/O-bound

Ráfagas cortas de CPU, mucha E/S.



El SO debe balancear para que ninguno "inunde" el sistema.

Métricas de rendimiento (performance metrics)

- **CPU Utilization:** porcentaje de tiempo que la CPU está ocupada (queremos que sea lo más alto posible).
- **Throughput:** cantidad de procesos terminados por unidad de tiempo.
- **Turnaround Time:** tiempo total desde que un proceso llega hasta que termina.

- **Waiting Time:** tiempo total que un proceso pasa esperando en la cola de ready.
- **Response Time:** tiempo desde que se envía una petición hasta que aparece la primera respuesta (clave en sistemas interactivos).

Inanición (Starvation)

- Ocurre cuando un proceso está listo para ejecutarse (*ready*) pero **nunca recibe CPU** porque siempre hay otros procesos que el scheduler elige antes.
- No es bloqueo (el proceso *podría* ejecutarse), sino que queda "esperando indefinidamente" por malas políticas de scheduling.

Algoritmos clásicos

FCFS (First Come First Served).



Un enfoque similar a FIFO (First In First Out)

Problema

- Supone que todos los procesos son iguales.
- Si llega un *megaproceso* que requiere mucha **CPU**, taponar a los demás procesos.

Solución

- Agregar prioridades al modelo.

Nuevo problema (Inanición/Starvation)

- Los procesos de mayor prioridad demoran infinitamente a los de menor prioridad, que nunca se ejecutan.

Posible solución

- Aumentar la prioridad de los procesos a medida que van "*envejeciendo*".



Cualquier esquema de prioridades fijas corre riesgo de inanición

Round Robin (RR).

La idea es darle un **quantum** a cada proceso, e ir alternando entre ellos.

¿Cúanto dura el quantum?

- Si es muy largo, en **SO** interactivos podría parecer que el sistema no responde.
- Si es muy corto, el tiempo de **scheduling + context switch** se vuelve una proporción muy importante del **quantum**.
- Por ende, el sistema pasa un porcentaje alto de su tiempo haciendo "mantenimiento" en lugar de trabajo de verdad.

Conbinación con prioridades

- Pueden estar dadas por el tipo de usuario (administrativas) o pueden ser "decididas" por el propio proceso, aunque este ultimo no suele funcionar.
- Van decreciendo a medida que los procesos reciben su quantum, para evitar inanición con otros.

SJF (Shortest Job First).

Esta idea para sistemas donde predominan los trabajos **batch**. Esta orientada a maximizar el **throughput**.

Duración total

Se puede predecir la duración del trabajo o al menos clasificarlo:

- menos de 10'.
- menos de 30'.
- menos de 60'.
- más de 60'.



Si se conoce las duraciones de antemano es óptimo.

Uso de CPU

Otra alternativa es pensar cuánto tiempo necesita para hacer **E/S** de nuevo.

Problema:

Comó saber cuanta CPU va a necesitar cada proceso.

Posible solución:

Usar información del pasado para predecir cuanta CPU necesita cada proceso.

Posible problema:

Si los procesos tiene comportamientos irregulares puede salir mal.

SRTF (Shortest Remaining Time First).

Una versión apropiativa del algoritmo de SJF.

Este algoritmo, el scheduler siempre selecciona el proceso cuyo tiempo restante en ejecución sea el más corto.

- Se debe conocer el tiempo de ejecución de antemano.
- Cuando llega un nuevo trabajo, su tiempo total se compara con el tiempo restante del proceso actual.
- Si el nuevo trabajo necesita menos tiempo para terminar que el proceso actual, éste se suspende y el nuevo trabajo se inicia.
- Ese esquema permite que los trabajos cortos nuevos obtengan un buen servicio.

Prioridades (con/sin desalojo).

- **Definición:**

A cada proceso se le asigna una **prioridad** y el scheduler elige siempre al proceso con mayor prioridad en estado *ready*.

Si varios procesos tienen la misma prioridad, se usa **First-Come, First-Served (FCFS)** entre ellos.

- **Relación con otros algoritmos:**

El **SJF (Shortest Job First)** es un caso particular de prioridades, donde la prioridad está dada por la duración estimada de la siguiente ráfaga de CPU (cuanto más corta, más alta la prioridad).

- **Variantes:**

Puede ser:

- **Preemptive:** el proceso en ejecución puede ser desalojado si llega otro con mayor prioridad.
- **Non-preemptive:** el proceso en ejecución continúa hasta terminar su ráfaga, aunque llegue otro más prioritario .

- **Problemas:**

- **Starvation (inanición):** procesos de baja prioridad pueden no ejecutarse nunca si siempre llegan procesos de alta prioridad .
- **Solución:** usar **aging**, que consiste en **incrementar gradualmente la prioridad de procesos que esperan mucho tiempo** .

Colas de multinivel o Múltiples colas (Multilevel Queue Scheduling).

- La **idea base** es que **existen varias colas de ready con quanta diferente**, separadas por categorías de procesos.
- Y su **objetivo** es minimizar el tiempo de respuesta para los procesos interactivos, suponiendo que los c ó mputos largos son menos sensibles a demoras.
- Ejemplo:
 - Cola 1: procesos interactivos.
 - Cola 2: procesos batch.
 - Cola 3: procesos de baja prioridad.
- A la hora de elegir un proceso la prioridad la tiene siempre la cola con menos quanta.
- Cada cola puede tener **su propio algoritmo** (FCFS, RR, SJF...).

- El **scheduler global** decide de qué cola se elige el próximo proceso (generalmente con prioridades fijas: la cola 1 siempre se atiende antes que la 2, etc.).
- **Problema:** los procesos de colas de baja prioridad pueden sufrir inanición (*starvation*).

Colas de multinivel con feedback (aging).

- Es una **extensión** de las colas multinivel.
 - La gran diferencia:
 - Los procesos **pueden cambiar de cola** según su comportamiento.
 - Ejemplo típico:
 - Proceso nuevo entra en la cola de más alta prioridad (quantum pequeño).
 - Si consume mucho CPU sin bloquearse, se lo "castiga" → se baja a una cola de menor prioridad.
 - Si espera demasiado (*starvation*), se lo "premia" → se sube de cola (**aging**).
 - Esto da un sistema más flexible: combina justicia con buen tiempo de respuesta para interactivos.
-

Scheduling en Tiempo Real

- Por lo general, uno o más dispositivos físicos externos a la computadora generan estímulo y la computadora debe reaccionar de manera apropiada a ellos dentro de cierta cantidad fija de tiempo.
- Los sistemas de tiempo real son aquellos en donde las tareas tienen fechas de finalización (*deadlines*).
- En general se usan en entornos críticos: si un *deadline* no se cumple, algo malo pasa.

Tiempo real duro

- Implica que hay tiempos límite absolutos estrictos que se deben cumplir.

Tiempo real suave

- Significa que no es conveniente fallar en un tiempo límite en ocasiones, pero sin embargo es tolerable.



En ambos casos, el comportamiento en tiempo real se logra dividiendo el programa en varios procesos, donde el comportamiento de cada uno de éstos es predecible y se conoce de antemano.

EDF (Earliest Deadline First).

- Una política posible consiste en correr el proceso más cercano a perder su deadline.
- Cuando un proceso puede ejecutar, debe anunciar sus requerimientos de deadline al sistema. Las prioridades deben ajustarse para reflejar el deadline del nuevo proceso.
- En teoría, EDF es óptimo ya que cada proceso puede cumplir su deadline y se maximiza el uso de CPU. En la práctica, no siempre es factible.

Scheduling con multiprocesamiento simétrico (SMP)

- Todos los procesadores/cores son **idénticos** y comparten **la misma memoria y recursos de E/S**.
- El sistema operativo puede planificar **cualquier proceso en cualquier CPU**.
- Se usa un **scheduler común** para todos los procesadores.
- Ventaja:
 - Flexibilidad → mejor balanceo de carga.
- Desafío:
 - **Contention (contención)** en acceso a memoria compartida.
 - Necesidad de sincronización (locks, semáforos).
- Cada uno de los procesadores se auto-planifica.

- Todos los procesos pueden estar en una cola común de procesos preparados.
 - También cada procesador pueden tener su propia cola privada de procesos preparados.
-

Scheduling con multiprocesamiento asimétrico (AMP)

- Consiste en que todas las decisiones sobre la planificación, el procesamiento de E/S y otras actividades del sistema sean gestionadas por un mismo procesador, el servidor maestro.
 - Cada procesador puede tener **roles distintos**.
 - Generalmente hay un procesador **maestro** que controla el scheduling, E/S y coordinación, y los demás **esclavos** ejecutan solo tareas que el maestro les asigna.
 - Ventaja:
 - Simplicidad (menos problemas de sincronización).
 - Desventaja:
 - Menos flexibilidad, el maestro puede ser cuello de botella.
-